

© Sumit Raj 2019

Sumit Raj, *Building Chatbots with Python*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-4096-0_4

4. Building Chatbots the Hard Way

Sumit Raj¹

(1) Bangalore, Karnataka, India

“Building Chatbots the Hard Way” is not too hard to learn. It’s the hard way of building chatbots to have full control over your own chatbots. If you want to build everything yourself, then you take the hard route. The harder route is hard when you go through it but beautiful and clear when you look back.

It is a rough road that leads to the heights of greatness.

—Lucius Annaeus Seneca

If you know Python well and have a bit of understanding on how to setup packages, etc., you won’t have any issues learning this chapter. If you are a developer, it should be easy for you. If you are a manager or non-technical person, you can still do the steps one-by-one as mentioned in each section and get things done. I strongly recommend everyone complete this chapter so that you learn core things about building chatbots.

This chapter not only teaches you to build chatbots from scratch but also shows you how core machine learning (ML) works with NLP with the help of Rasa NLU. As mentioned in the first chapter on this book, it is always good to have decision trees when you are building chatbots. In this chapter, we’ll barely be using any rules, but ML is not at that stage yet to be 100 percent reliable. So, this decision is based completely on your use-case and whether you want to apply some business logic along with your ML models or not. Sometimes, your ML model may work so well that you don’t need any heuristics at all. But in my experience, when you go on to sell your chatbots or commercialize them, you have to be a little careful. No functionality is better than having a functionality that doesn’t make sense.

We’ll be using an open-source library called Rasa NLU for learning how to build chatbots from scratch without using any cloud services like Dialogflow, Watson, wit.ai, etc. Remember Rasa NLU is a very sophisticated library and has lots of features in it. We are only going to touch upon the concepts and features that are important for us to build our chatbot.

What Is Rasa NLU?

Rasa NLU is an open-source NLP library for intent classification and entity extraction in chatbots. It helps you build and write custom NLP for your chatbots.

There are two parts of Rasa that we’ll be covering in this chapter.

- **Rasa NLU:** With Rasa NLU we'll be learning to prepare our training data for our chatbot, writing configuration files, choosing a pipeline and training the model. Lastly, we will be predicting the intent of a text using our model. We'll also be learning how to parse entities using Rasa NLU.
- **Rasa Core:** In the second part we'll be learning to train the **Rasa Core** dialog management model to prepare responses back to the user. This section becomes very important when you have a variety of intents in your chatbot and their follow-up questions or responses. Instead of writing lots of conditions in our decision tree and spending hours debugging it in case of a big enterprise-level application, it's better to teach the model to create responses. It'll be interesting to see how well our trained model performs in doing that. We can't just spit out anything to the user in the form of text; it should make sense.

Why Should I Use Rasa NLU?

Rasa NLU is not just any other library with a bunch of methods to do some things. It has the capability of building almost any kind of chatbot that you can imagine. Rasa brings you the magical capability of training the machine to understand the meaning of a text rather than you writing rules to understand it.

Let's look at the following points on why we should use Rasa NLU:

- Rasa NLU is an actively maintained project and has a good community to support it.
- If we don't want to share our user's sensitive data with a third party, we must use open-source tools like Rasa NLU to build chatbots from scratch. This way all the data remains and gets processed on our own servers.
- Depending on third-party services for training your data and finding the intents of user utterances will require you to call APIs that may not always be reliable. What happens to your chatbot application if their server is down?
- Using Rasa NLU for building chatbots will give you full command over your chatbots. You can train, tune, and optimize it the way you want with the data you want. With Rasa NLU we can experiment with which ML algorithm works best for our dataset rather than depending on a fixed algorithm.

Diving Straight Into Rasa NLU

In this section we'll dive straight into the hands-on part and try installing the Rasa stack and start working on building our ML models by training our data. We'll be using some of the cooler open-source libraries to make our lives easier.

Installing Rasa

To install Rasa, run the following pip command that we tried in previous chapters for installing spaCy. Note that we'll be using Rasa version 0.13.2.

```
pip install rasa-nlu==0.13.2
```

Rasa NLU has multiple components for classifying intents and recognizing entities. Different components of Rasa have their own sets of dependencies.

When we train our model, Rasa NLU checks that all the required dependencies are installed. If you want to install the full requirements needed to entirely use the Rasa library, you can execute the following steps:

```
git clone https://github.com/RasaHQ/rasa_nlu.git #Clone the repo
cd rasa_nlu #Get into the rasa directory
pip install -r alt_requirements/requirements_full.txt #Install full requirements
```

The first step may take some time. Be as patient as earth until it completes.

Deciding a Pipeline in Rasa

Pipeline is nothing but a set of algorithms to be used to train your model. Rasa NLU has two widely used pipelines called `spacy_sklearn` and `tensorflow_embedding`. Let's learn a bit about both.

`spacy_sklearn`

- *spacy_sklearn* pipeline makes use of pre-trained word vectors from either GloVe algorithm or an algorithm developed by the Facebook AI team called fastText.
- *spacy_sklearn* works amazingly well in situations where, suppose you have an utterance like, "What is the weather in Boston?" When we train our model on the same utterance example and then ask it to predict the intent for, "What is the weather in London?" our model is now intelligent enough to know that both the words "Boston" and "London" are similar, and they belong to the same intent.
- This pipeline is very useful with small sets of data.

`tensorflow_embedding`

- *tensorflow_embedding* pipeline doesn't make use of any pre-trained word vectors like *spacy_sklearn*, but it adjusts itself as per our own provided dataset.
- The good thing about *tensorflow_embedding* pipeline is that our word vectors will be as per our domain.
- To explain how *tensorflow_embedding* works with an example, in English language, the word "play" may be closely related to "a sport" or "an activity of enjoyment or recreation," and it may seem it very different from the words "an act." In a theater domain, "play" and "an act" are closely related, where "play" means "a form of literature written by a playwright," and it is very necessary to tell our model to learn specific to our domain and not get confused due to some pre-trained model.

Training and Building a Chatbot From Scratch

If you have gone through Chapter 3 of this book, you must be familiar with the “Food Ordering Chatbot” we built using Dialogflow. You must be aware of intents, entities, and responses back to the end user by a chatbot.

Similarly, we are going to take a use-case of a chatbot and build it from scratch in this chapter. You need not necessarily use the same example. Feel free to find a use-case of your own, follow the steps in this chapter, and build your own chatbot at the completion of this chapter.

We are going to build a horoscope bot that understands user queries and tells them their horoscope for the day. So, let's get started.

Building a Horoscope Bot

In this example of building a chatbot completely on our own using the open-source library Rasa NLU, we are going to build a Horoscope Bot. Let's decide the scope of this chatbot and see what it does and can do.

- The Horoscope Bot should be able to understand greetings and reply with a greeting.
- The bot should be able to understand if the user is asking for a horoscope.
- The bot should be able to ask the horoscope sign of the user if the user doesn't provide it.
- If the user doesn't know his/her horoscope sign, the bot should ask the user's date of birth (DOB) and find it for them.
- The bot should subscribe/unsubscribe the user to get the daily horoscope.
- The bot should learn from existing responses to formulate a new response.
- The bot should be able to handle spelling mistakes by the user.

It is pretty simple what our bot is supposed to do here.

Possible intents

- Greeting Intent: User starting with a greeting
- Get Horoscope Intent: User asking for horoscope
- User's Horoscope Intent: User telling the horoscope sign
- User's DOB Intent: User telling his/her DOB
- Subscription Intent: User asking for subscription

Depending on the way you design your bot and write code, you may or may not need all of the above intents or you may need to define some extra intents as well to cover all the use-cases we want our bot to do. As we move forward in this chapter, we'll try to build the bot that does the basic task of giving a horoscope; later as a part of the exercise, we can add more use-cases to it.

Well, this is pretty much what we have been doing so far to recognize the possible intents based on a simple possible conversation between a bot and a user.

Conversation Script Between the Horoscope Bot and the User

Let's create a possible conversation script between our chatbot and the user. As mentioned in the first chapter, writing a conversation script is like preparing a software requirement document before building the software. It may change later but gives you an overall view of what's going to happen in the final chatbot application.

The following is what our conversation between the user and the bot looks like:

- **User:** Hello
- **Horoscope Bot:** Hello! Hope you are doing well.
- **User:** I want to know my horoscope.
- **Horoscope Bot:** Sure, may I know your horoscope sign?
- **User:** Capricorn
- **Horoscope Bot:** Sorry, did you mean Capricorn?
- **User:** Yes.
- **Horoscope Bot:** Alright, Here is your horoscope for today. Chances are that you'll feel upbeat and positive today. Share this energy with others who feel less than content. Your attitude can have a profound effect on friends, partners, spouse, and children. Neighbors and extended family can pick up on your spirit without you even realizing it. If someone is hurt or angry, be encouraging. Everything will eventually work out.
- **Horoscope Bot :** Would you like to subscribe for the daily updates? If yes, just say subscribe.
- **User:** Yes, please subscribe.
- **Horoscope Bot:** Great, you are subscribed now. Have a good day.

This conversation is just to have a fair idea of how our chatbot conversation is going to look. We may or may not want to change the flow of the conversation based on our needs.

We can have our chatbot model itself trained to prepare a valid response instead of writing a bunch of if...else statements.

Preparing Data for Chatbot

Rasa NLU has multiple ways of defining the intents and their entities on our custom data. It supports data in markdown, in JSON as a single file, or as a directory containing multiple files.

We are going to discuss the most difficult, but highly scalable, method first. Creating JSON files is difficult by hand but programmatically very easy and scalable.

Creating Data for Model in JSON Format

The JSON format of the data that Rasa NLU expects has a top-level object called **rasa_nlu_data**, with the keys **common_examples**, **entity_synonyms**, and **regex_features**.

The most important one with which we are going to be working is **common_examples**. The following is the skeleton form of how our JSON data is going to look:

```
{
  "rasa_nlu_data": {
    "common_examples": [],
    "regex_features" : [],
    "entity_synonyms": []
  }
}
```

The **common_examples** key in our JSON data is the central place that'll be used to train our model. We will be adding all our training examples in the **common_examples** array.

regex_features is a tool to help the intent classifier recognize entities or intents and improve the accuracy of intent classification.

Let's start writing our JSON file. Let's call it **data.json**.

1. Create a folder called **horoscope_bot**.
2. Change the current working directory to **horoscope_bot**.
3. Start Jupyter Notebook #jupyter notebook.
4. Create a new folder called **data**.
5. Click on the data folder and go to "Text File" under New menu in Jupyter Notebook.
6. Click on the name of the file created and change the name to **data.json** and write your intents for your chatbots.

For Steps 5 and 6, feel free to use your favorite editors like Sublime, Notepad++, PyCharm, etc., to work with the JSON file.

This following is what my **data.json** under **data** folder looks like:

```
{
  "rasa_nlu_data": {
    "common_examples": [
      {
        "text": "Hello",
        "intent": "greeting",
        "entities": []
      },
      {
        "text": "I want to know my Horoscope",
        "intent": "get_horoscope",
        "entities": []
      },
      {
        "text": "Can you please tell me my horoscope?",
        "intent": "get_horoscope",
        "entities": []
      },
      {
        "text": "Please subscribe me",
        "intent": "subscription"
      }
    ]
  }
}
```

```
    ],
    "regex_features": [],
    "entity_synonyms": []
  }
}
```

Well as you can see, it looks very clumsy to prepare this by hand. You must be remembering the nice and easy method we had in Dialogflow. So, let's check a cool and interesting tool for creating training data in the format that Rasa expects. It was created by Polgár András, and it's also pretty much good for inspecting and modifying existing data that we prepared earlier. This tool saves a lot of time if you are working on a small project where you have to create the data by hand. It's always a good idea to visualize the data in any application you are building that is completely data-driven.

So, just save the earlier **data.json** file we created until we extend the data collection using a better method.

Visualizing and Modifying Rasa's JSON Data Format

In this section we'll make use of a tool called Rasa NLU trainer to visualize our data. (i.e., the data that we have created so far). This tool helps us in annotating the data as well. If you remember when the Dialogflow interface was explained in Chapter 3, it was so easy to define entities, their names, and the types. We are going to do the same using an open-source tool.

Rasa NLU trainer is a very nice and handy tool to edit our training data right from our browser itself. Handling JSON data is tricky and also can lead to errors. With this handy tool we can easily add more examples to our training data or edit the existing ones. It saves a lot of time from manually annotating the data. `rasa-nlu-trainer` is a javascript-based tool, so we would need to install `node.js` to run this tool on our system. It doesn't take more than 5 minutes to do that. Let's get this set up by following these steps:

1. Go to <https://www.npmjs.com/get-npm> and download `node.js`.
2. Install the package on your system as guided on the website. Once installed, go to a fresh terminal/command line interface on your system and type "npm" to see if it works.

I have installed LTS version 8.11.4. Once installed, run the following command to install `rasa-nlu-trainer`:

```
sudo npm i -g rasa-nlu-trainer
```

After the successful installation this command, you'll see logs similar to the following:

```
[fsevents] Success: "/usr/local/lib/node_modules/rasa-nlu-trainer/node_modules/fsevents/lib/binding/
Pass --update-binary to reinstall or --build-from-source to recompile
npm WARN slick-carousel@1.8.1 requires a peer of jquery@>=1.8.0 but none is installed. You must inst
```

```
+ rasa-nlu-trainer@0.2.7
added 492 packages in 10.14s
```

Even if your message doesn't look like this, as long as it doesn't throw any errors then don't worry. We'll get to know in a moment if our `rasa-nlu-trainer` is successfully installed and works well.

Let's go to our data folder that we created earlier in our terminal and run the following command:

```
rasa-nlu-trainer
```

Typing this command will crank up a local server on port 55703 and open it in the default browser. It will look something like [Figure 4-1](#).

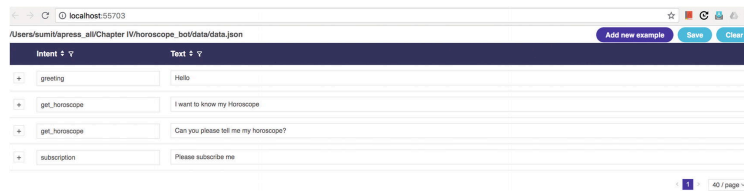


Figure 4-1 rasa-nlu-trainer in localhost

As you can see, all of our existing data from `data.json` is picked by this amazing tool for us to delete or edit and we can also add new examples from here and it will keep on extending the `data.json` file.

I would suggest that you add more data to your intents for better training of the model. You can get this `data.json` in the source code zip or repo provided by the publisher, if you are trying to build the same chatbot explained in this chapter.

Just as we selected the entities within utterances to define them in Chapter 3 using Dialogflow, we can do the same using this tool and give names to our entities for parsing later. So, click on the toggle button of the example, select the text, and add an entity by giving it a name.

I have added five to six utterance examples in each of the intents I have defined. The more examples we add, the better it is for the model to be trained and provide better accuracy.

If you look at the `data.json` file now, it will have more examples automatically added to it. So, go ahead and verify your `data.json` file to see if you have all your added examples from **rasa-nlu-trainer** UI.

You'll also notice in the `data.json` file that the entities you might have defined using the `rasa-nlu-trainer` UI are captured in the **common_examples** list as having **start** and **end** keys, which tells the model at what point the particular entity value starts in the example and when it ends.

The same dictionary object also depicts the value of the entity and the name of the entity we defined. For our example, it looks like the following:

```
{
```



```

"text": "19-01",
"intent": "dob_intent",
"entities": [
  {
    "start": 0,
    "end": 2,
    "value": "19",
    "entity": "DD"
  },
  {
    "start": 3,
    "end": 5,
    "value": "01",
    "entity": "MM"
  }
]
}

```

Training the Chatbot Model

In this section we are going to train a model on the data we prepared. As we used Jupyter Notebook for our file creation and management, we will be creating a new .ipynb and start writing our Python code to train our model by choosing one of the pipelines we discussed earlier in this chapter.

Creating a Configuration File

Let's create a JSON file again in the same way we created one earlier using Jupyter and name it **config.json**. Let's keep it outside our data folder (i.e., in the horoscope_bot that is our project directory).

Add the following configuration to it:

```

{
  "pipeline": "tensorflow_embedding",
  "path": "./models/nlu",
  "data": "./data/data.json"
}

```

As you can see, there are some important configuration parameters done in our config.json file. Let's try to understand them.

- **pipeline:** Pipeline is going to specify what featurizers or feature extractors are going to be used to crunch text messages and extract necessary information. In our case, we are using *tensorflow_embedding*.
- **path:** path is essentially the directory where we keep our model after the training. We are going to keep our model in the /models/nlu folder.
- **data:** data is the path we need to specify; it's basically where our training data sits.

As we are done with our `config.json` file, let's move on to some Python code to train our ML model.

YAML Configuration

You can also use `.yaml` file as config files like below. You can get the example `config.yaml` files in github repo.

- **Example 1:**

```
language: "en"
pipeline: "tensorflow_embedding"
```

- **Example 2:**

```
language: "en"
pipeline:
  - name: "nlp_spacy"
  - name: "tokenizer_spacy"
  - name: "intent_entity_featurizer_regex"
  - name: "intent_featurizer_spacy"
  - name: "ner_crf"
  - name: "ner_synonyms"
  - name: "intent_classifier_sklearn"
```

All incoming messages are processed as per the sequence of components defined. The defined components are run in a sequential manner, one by one, and hence are called the processing pipeline. Different components are used for different purposes, such as entity extraction, intent classification, pre-processing, etc.

The benefit of such a format is that we can specify the predefined pipelines by Rasa in a clear manner.

Writing Python Code to Train the Model and Predict

Let's open up a new `.ipynb` file and start writing our code. Let's name the `ipynb` as `rasa-nlu.ipynb`. Make sure you have already successfully installed `rasa-nlu==0.13.2` for the Python version you are using.

The following is what our code looks like to use our `data.json` and `config.json` in Python and train a model using `tensorflow_embedding` pipeline.

```
from rasa_nlu.training_data import load_data
from rasa_nlu.model import Trainer
from rasa_nlu import config
from rasa_nlu.model import Interpreter

def train_horoscopebot(data_json, config_file, model_dir):
    training_data = load_data(data_json)
    trainer = Trainer(config.load(config_file))
    trainer.train(training_data)
    model_directory = trainer.persist(model_dir, fixed_model_name = 'horoscopebot')

def predict_intent(text):
    interpreter = Interpreter.load('./models/nlu/default/horoscopebot')
    print(interpreter.parse(text))
```

In the first section of the code, we import all the necessary libraries needed from `rasa_nlu` package. Then we define two methods called `train_horoscopebot` and `predict_intent`, where the first method trains the model given the data, `config_file`, and `model_directory` (place to store the models) and `predict_intent` method uses the **Interpreter** model from `rasa_nlu` to load the pre-trained model files and gives the user the ability to predict any new text examples.

Training the Model

We run the below snippet to call our `train_horoscopebot` method with the respective parameters

```
train_horoscopebot('./data/data.json', 'config.json', './models/nlu')
```

After running this code in our `rasa-nlu.ipynb`, we will get an output like this:

```
Epochs: 100%|██████████| 300/300 [00:01<00:00, 175.69it/s, loss=0.075, acc=1.000]
```

The code for training the chatbot model will create the models folder, which you can see using Jupyter or using your file explorer or finder app. It will create a bunch of index, meta, and pickle files at the model directory destination we provided.

Predicting From the Model

Let's call the `predict_intent` method by passing a text to see how our trained model performs.

```
predict_intent("I am looking for my horoscope for today. I am wondering if you can tell me that.")
```

The method itself prints the output. For the above text, my output looks like the following:

```
INFO:tensorflow:Restoring parameters from ./models/nlu/default/horoscopebot/intent_classifier_tensor
{
  "intent": {
    "name": "get_horoscope",
    "confidence": 0.9636583924293518
  },
  "entities": [],
  "intent_ranking": [
    {
      "name": "get_horoscope",
      "confidence": 0.9636583924293518
    },
    {
      "name": "dob_intent",
      "confidence": 0.03462183475494385
    },
    {
      "name": "greeting",
      "confidence": 0
    },
    {
```

```

        "name": "subscription",
        "confidence": 0
    }
],
"text": "I am looking for my horoscope for today. I am wondering if you can tell me that."
}

```

Wow! Isn't it magical? Our model has predicted this text with a confidence of 96 percent. You can see in the provided ipynb file that our model does well in predicting other intents as well. This is the power of tensorflow and ML overall. Needless to say, rasa_nlu library makes it so easy to believe. So, it's time for you go back in retrospection, and if you remember Chapter 3 of this book, then you must remember whenever we used to add a new example, the Dialogflow used to re-train the model. It was actually doing the same as we just did, behind the scenes. We couldn't have changed the model or tuned any parameter there, but we can do that all now with full control.

Now that we have successfully built and trained a model using tensorflow and tested it as well, we'll move on to the next topic of Dialog Management. I would request you test all the scenarios your bot might be facing so that you know the points where your model is not performing well and, accordingly, you can add more data or tune the parameters if needed.

Also, remember that you only need to re-train the model whenever there is a change in the training data. If there is no change in the training data, we can load the existing training model to keep predicting on new examples.

Dialog Management Using Rasa Core

In this section we are going to get our hands dirty by training another model for Rasa Core dialog management. Remember, at this point we have a model ready to predict the intent of the text, we can write some Python code to formulate responses, and we can reply back to the customer. But what if we want to add more intents to our bot? Is that scalable in the case of a bigger application with lots of features? The answer is no. Here dialog management from Rasa Core comes to the rescue.

If you have ever tried to use any bot on any platform, you must have seen it failing in certain conditions. Yes, we all have been there and it still exists as today's bot fails to manage the contexts of the conversation miserably or follow the suit of conversation. With the help of Rasa Core's ML-based dialog framework we can fix this issue easily. Rasa Core is well-proven for enterprise-level applications and used by thousands of developers because it is production-ready, easy to use and extend, and, most importantly, open source.

Understanding More on Rasa Core and Dialog System

Before we actually move to the coding part of Rasa Core for our dialog management model, it's really important to understand why and where this is coming from.

We'll try to understand how we have been doing things so far for building the chatbots and how this is going to be changed forever.

Let's take an example:

If we were to build a simple chatbot that would help users book flight/bus/movie/train tickets, the easiest way would be to create a state machine or decision trees, write a bunch of if...else, and it would be done. This would work but wouldn't scale. If a customer is having a good experience initially with something, they want to use it more. By some heuristics, we can show a chatbot to be intelligent but not for very long. When the control flow of the code goes from try block to except block, we start scratching our heads.

Figure 4-2 is a simple representation of how a state machine for building this chatbot may look.

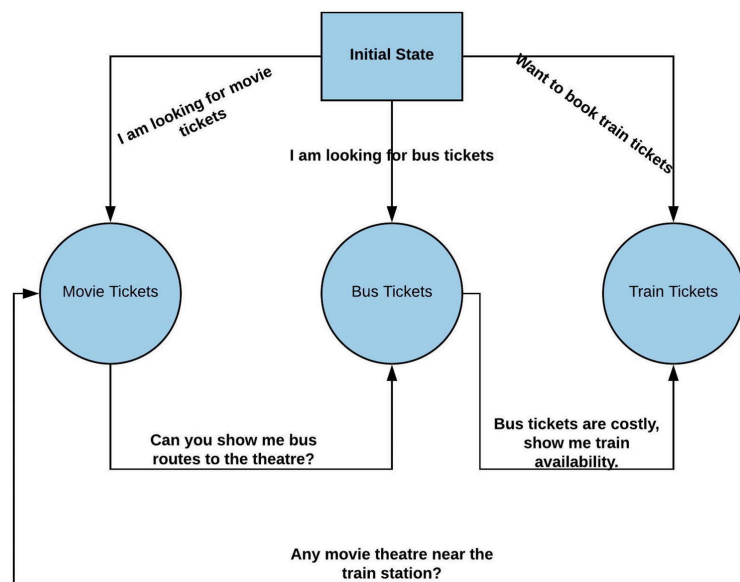


Figure 4-2 State diagram representation for ticket booking chatbot

If we look at our state diagram, it may work for a normal conversation where the user is looking for movie, bus, or train tickets or wants to book a bus ticket after asking for movie tickets. What if the user asks for both bus and movie tickets together? You may say that we can add a couple more if...else statements in our already nested code to handle that. If you are a good developer, it won't take you much time to write an in-out from a state machine or extend your decision tree. But think of the situation when these conditions start growing exponentially and you have to keep adding cases to handle that and they start interfering with each other as well.

Our brain works in a way where we learn and relearn. If a kid doesn't know what a fire is going to do to them, they touch it, but when it hurts them, they do not do it again. They reinforce a fact that it's harmful. It works similarly in the case of rewards—when you do something and get something, you associate a fact that doing something brings rewards or better rewards and then you intend to do that again. This is called reinforcement learning in ML, where a machine learns how to behave in a

particular situation by performing actions and understanding the results. Reinforcement learning is sometimes not the best approach, such as in situations where data is not sufficient to be learned, data quality is not good to learn the reward scenarios, etc.

Figure 4-3 is a diagram to help you understand how Rasa Core fits well within the Rasa Stack and its role along with Rasa NLU, which we have been learning so far.

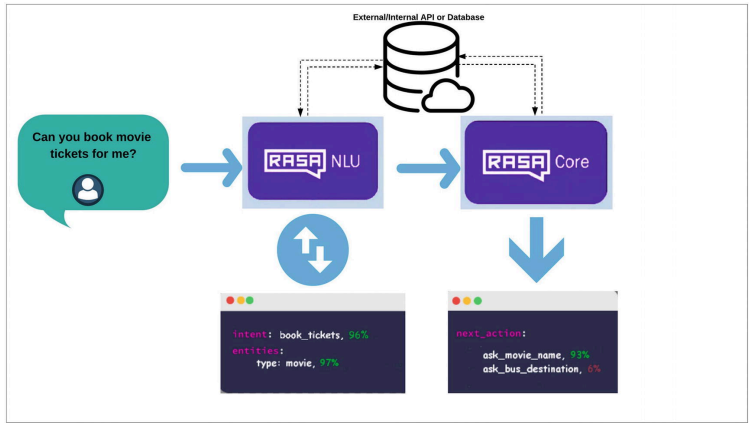


Figure 4-3 Diagram representing the working of Rasa NLU and Rasa Core

With Rasa Core, we can specify all of the things our bot is supposed to say or do. These are called *actions*. One action might be to say a greeting to the user or to query a database, or it could be to retrieve some information using some web service or API.

Rasa Core provides the facility to train our probabilistic model to predict what action has to be taken based on the historical conversation of the user or users. Imagine doing it without an ML model and writing hundreds of use-cases to figure out what response or action should be sent or executed. In simple terms, Rasa Core solves this problem for you.

Let’s look at the side-by-side user vs. bot conversation that we built in Chapter 3 while building an OnlineEatsBot.

User	OnlineEatsBot
Hello OnlineEats	Hello! How may I help you?
I am looking to order food	Sure, What would you like to order today?
One chicken burger	Done. Your final amount is \$3.99
Thank you	Thanks a lot

This conversation is pretty much simple and can be done without any ML model as well, even though we did use a trained model to recognize the intent and entities in it.

Now, the interesting question that will come to your mind is, “What if I want my bot to be able to change the order? What if I want my bot to be

able to remove or add more quantity of the food items?” So, as discussed, add more use-cases, more complexity, more intents and their utterances, more if...else in code to handle corner-cases, but when you are building a chatbot for a business you have to scale it to generate more revenue through it. All software systems do that, and those who don’t fail to survive. The bottom line is, we can’t keep changing and deploying the code.

So, instead of handling all the cases as they occur and repeatedly doing it, we can take the ML route to tackle this problem, with the help of Rasa Stack, which will tell us what the bot should do next based on the user’s context and the state of the conversation. Since the model learns itself based on the context of the previous conversation data, it becomes easier for the bot to keep the conversation more natural and user-friendly rather than randomly choosing from four to five fixed sentences.

Rasa recommends users with little or no data use interactive learning. We’ll get to know more about **interactive learning** later in this chapter.

Before we actually start using Rasa Stack to write the core part of our bot, first we need to understand a few concepts.

Understanding Rasa Concepts

It’s really important that we understand a few specific concepts related to Rasa before actually trying to use them in code completely. In this section we’ll be learning a few important and very useful concepts of Rasa NLU. Make sure you understand these concepts completely, as we will be using them in building our first in-house chatbot using Rasa’s domain file format. If we do not understand what these concepts mean, then it’ll be difficult to take the next step.

Action

As the name suggests, it’s a specific action that can be taken. As per Rasa documentation, it says, “*Next action to be taken in response to a dialog state.*”

For example, if a user asks the horoscope for today, our bot could execute the “GetTodaysHoroscope” action. Let’s see how a “GetTodaysHoroscope” action would look in form of a code.

```
from rasa_core.actions import Action
from rasa_core.events import SlotSet
class GetTodaysHoroscope(Action):
    def name(self):
        return "get_todays_horoscope"
    def run(self, dispatcher, tracker, domain):
        # type: (Dispatcher, DialogueStateTracker, Domain) -> List[Event]
        user_horoscope_sign = tracker.get_slot('horoscope_sign')
        """Write your logic to get today's horoscope details
        for the given Horoscope sign based on some API calls
        or retrieval from the database"""
        return [SlotSet("horoscope_sign", user_horoscope_sign)]
```

The **name** method returns the name of the Action that we'll refer to in the domain file as custom method name.

The **run** method does the main job for getting the action done—i.e., the core business logic resides here. As you can see, it takes three parameters: *dispatch*, *tracker*, and *domain*.

Let's understand these parameters one by one:

- **dispatcher:** the dispatcher is used to send messages back to our users. We can use `dispatcher.utter_message()` to achieve the same.
- **tracker:** the state tracker for the current user. We can access slot values using `tracker.get_slot(slot_name)`, and to get the most recent user message we can use `tracker.latest_message.text`.
- **domain:** the bot's domain. We'll discuss in more detail about the domain later in the chapter.

Noterun method returns a list of Events instances.

Slots

Slots are the ones who make the bot responsible for working like a human. Slots are like a storage space where the information given by the user can be stored or a pre-fetched information from a database or an API can also be used.

There are different slot types for different use-cases:

For example, in our use-case where we want to build a HoroscopeBot, we may want to use the slot type as **text** for the **horoscope_sign** provided by the user.

Depending on what type of slot you want to keep, Rasa provides some predefined slot types.

Other than text, Rasa has the following slot types:

- **Boolean** ⇒ Used For True/False
- **Categorical** ⇒ Used for situations where you have to pick one of some values
- **Float** ⇒ Used for continuous values
- **List** ⇒ Used for list of values
- **Featurized** ⇒ Used for internal value storage that doesn't affect the conversation

Templates

Templates is a word you must have heard in your life at least once, while finding a template for sending an e-mail, for preparing a document, for building a portfolio website, or to follow a process.

Templates in Rasa are used for utterances. An utterance template contains a set of preset text to be sent to the user when some action is triggered. By having the same name of the action as the utterance or by an

action with custom code, we can send our formatted message in templates to the user.

A simple representation of templates inside the domain file may look like the following:

```
templates:
  utter_greet:
    - "hello {name}!" # name can be filled by a slot of same name or by custom code
  utter_goodbye:
    - "goodbye"
    - "take care bye" # multiple templates allow the bot to randomly pick from them
  utter_default:
    - "Sorry, I didn't get that."
```

Now, that we have gone through the three concepts of actions, slots, and templates and we already know what intents and entities are as part of our learning from Chapter 3, we are now ready to dive deep into Rasa and start writing code for our first chatbot.

Creating Domain File for the Chatbot

The first and foremost task to do while building a chatbot using Rasa stack is to create a domain file.

As per the documentation of Rasa, “The Domain defines the universe in which your bot operates. It specifies the intents, entities, slots, and actions your bot should know about. Optionally, it can also include templates for the things your bot can say.”

Now, you know why we had to be prepared for this situation previously by understanding Rasa core concepts.

Let's create a DefaultDomain file with a YAML definition. Rasa uses .yaml files to define the domain format.

Originally YAML supposedly meant Yet Another Markup Language, depicting its use as a markup language, but it was changed later to be understood as YAML Ain't Markup Language, a recursive acronym, to differentiate its purpose as data-oriented rather than a document markup language.

Now, let's go back to our rasa-nlu Jupyter Notebook directory and start creating the files. Note that we can write all our code in separate files using a command line and use an editor to edit it. I find Jupyter Notebook more interactive and easily accessible to browse through the files. Whatever you are comfortable with, go with that, but it is good to know most of the features that Jupyter Notebook provides.

Go to the main horoscope_bot directory and create a file, let's call it horoscope_domain.yaml.

The following is the content of horoscope_domain.yaml for our bot:

```

slots:
    horoscope_sign:
        type: text
    DD:
        type: text
    MM:
        type: text
    subscribe:
        type: bool
intents:
    - greeting
    - get_horoscope

    - subscription
    - dob_intent
entities:
    - horoscope_sign
    - DD
    - MM
    - subscribe
    - dob_intent
templates:
    utter_greet:
        - 'Hello! How are you doing today?'
    utter_ask_horoscope_sign:
        - 'What is your horoscope sign?'
    utter_ask_dob:
        - 'What is your DOB in DD-MM format?'
    utter_subscribe:
        - 'Do you want to subscribe for daily updates?'
actions:
    - utter_greet
    - utter_ask_horoscope_sign
    - utter_ask_dob
    - utter_subscribe
    - get_todays_horoscope
    - subscribe_user

```

As you can see, the domain file is comprised of five important parts: `intents`, `entities`, `slots`, `templates`, and `actions`, which we discussed earlier.

Note that for every template, there is an `utterAction` defined, such as `utter_greet`, `utter_ask_horoscope_sign`, and `utter_ask_dob`, we must have a template defined in the template section with the same name.

As you can see in our example, there are mainly five actions defined, where the first three actions are just for uttering a template text to the user, but the last two actions require us to either retrieve the data from the database or make an API call to get the horoscope for the day and return it back to the user.

In the case of `subscribe_user` action as well, we need to do an operation where we add the current user to the subscribe list in the database. These

user-defined actions are called custom actions. To have such custom actions, we need to write what the bot is supposed to do when these actions are triggered.

In the next section we are going to learn how to write custom actions.

Writing Custom Actions of the Chatbot

As we know, whenever an `UtterAction` is triggered, our bot will respond back with the utter text defined in the template for that Action. But what happens when some custom action is triggered? In this section we'll write Python code to create custom actions, which we can use to make API calls or pretty much any kind of thing that you can do with Python.

Let's create a new file called `actions.py` in our project directory (i.e., in our case inside `horoscope_bot` folder).

```
from __future__ import absolute_import
from __future__ import division
from __future__ import print_function
from __future__ import unicode_literals
import requests
from rasa_core_sdk import Action
from rasa_core_sdk.events import SlotSet

class GetTodaysHoroscope(Action):
    def name(self):
        return "get_todays_horoscope"
    def run(self, dispatcher, tracker, domain):
        # type: (Dispatcher, DialogueStateTracker, Domain) -> List[Event]
        user_horoscope_sign = tracker.get_slot('horoscope_sign')
        base_url = http://horoscope-api.herokuapp.com/horoscope/{day}/{sign}
        url = base_url.format(**{'day': "today", 'sign': user_horoscope_sign})
        #http://horoscope-api.herokuapp.com/horoscope/today/capricorn
        res = requests.get(url)
        todays_horoscope = res.json()['horoscope']
        response = "Your today's horoscope:\n{}".format(todays_horoscope)
        dispatcher.utter_message(response)
        return [SlotSet("horoscope_sign", user_horoscope_sign)]
```

As we can see, there are two methods in our action called `GetTodaysHoroscope`. The first method **name** just returns the name of the action. The other method is **run** and, as discussed earlier, is the method that actually does the task by executing the business logic that we write.

In our method we are making use of an open-source API with code hosted on github [<https://github.com/tapasweni-pathak/Horoscope-API>]

The API url looks like this:

<http://horoscope-api.herokuapp.com/horoscope/today/capricorn>

which returns the data in JSON format:

```
{
  "date": "2018-08-29",
  "horoscope": "You will be overpowered with nostalgia and may long to get in touch with old pals. A
```

```
"sunsign": "capricorn"
}
```

As you can see in the **run** method, we convert the response from the API to Python JSON object and then access the 'horoscope' key from the JSON to get the actual horoscope. After getting the actual horoscope from the JSON, we formulate a response and dispatch it back to the user using the `dispatcher` object and its method, `utter_message`.

At the end we set the slot using `SlotSet` method. `SlotSet` is like saving the variables that you figured out from user's responses so that you can use them any time in your code during the flow of the conversation.

Note Using the above API we can get today's horoscope by providing the horoscope sign. You are free to use your own API or database. You just need to replace the API call with some other source that you want to use.

Just like we added the `GetTodayHoroscope` action in the `actions.py` file, we'll also add `SubscribeUser` action as well. We are not going to use any database to store the user subscription preferences, but when you are building a chatbot for real users, you may have to have `user_ids` that you can link with their subscriptions in the database.

The following is how the `SubscribeUser` action looks:

```
class SubscribeUser(Action):
    def name(self):
        return "subscribe_user"
    def run(self, dispatcher, tracker, domain):
        # type: (Dispatcher, DialogueStateTracker, Domain) -> List[Event]
        subscribe = tracker.get_slot('subscribe')
        if subscribe == "True":
            response = "You're successfully subscribed"
        if subscribe == "False":
            response = "You're successfully unsubscribed"
        dispatcher.utter_message(response)
        return [SlotSet("subscribe", subscribe)]
```

Like this, we can write as many custom actions as needed.

The next step is the data. Rasa's dialog management model is trained on actual conversations that users and the chatbot do. The important point here is that those conversations have to be converted into a story format.

A story is nothing but an actual conversation between a user and a chatbot where user inputs are converted into intents and entities while responses being returned from the chatbot are treated as actions that the chatbot is supposed to trigger when it's required.

One of the examples of how a real conversation between a user and a chatbot may look as a story is given in the table below.

Scenario I

User	HoroscopeBot
Hello there!	utter_greet
I want to know my horoscope for today	utter_ask_horoscope_sign
My sign is Capricorn	actions.GetTodaysHoroscope
Can you subscribe me for updates?	actions.SubscribeUser
Scenario II	
User	HoroscopeBot
Hello there!	utter_greet
I want to know my horoscope for today	utter_ask_horoscope_sign
I don't know my sign	utter_ask_dob
12-12	actions.GetTodaysHoroscope

We haven't yet covered the scenario in the code where the user doesn't know his horoscope sign but knows his DOB. Here, our code needs some modification to get the DATE and MONTH entities when horoscope_sign value is not found.

We could use DD-MM values to check the horoscope sign and then call the GetTodaysHoroscope method explicitly or train the model in that way.

Data Preparation for Training the Bot

It's always important to have good quality data before doing any kind of ML. For training our chatbot we also need data; the conversation between a user and a chatbot is the data on which we need to train our models. Sometimes it becomes difficult to find a dataset freely on the web that fits our own need.

We should spend the time we need to get the data gathered. We can ask our friends and family to provide us the sample conversation text of how they would interact with a kind of bot you are building. Some people create sample example apps for the same and crowd-source the data. So, better the data, better the model, and better the chatbot responses.

When it comes to preparing the data, Rasa leaves no stone unturned and comes with a cool feature called **interactive learning**. It helps you in generating story data easily and also trains the dialog management model as we keep adding the story data. You can call it real-time ML training. So, as we keep on adding our story data, we get to know if our model is producing correct output or not. Most importantly we get to see if the model is improving or degrading when we add new stories. In most cases it will

get better because we'll be kind of doing the reinforcement learning where we tell the ML model to unlearn and relearn—pretty much what human beings do.

Creating Story Data

As we know, story data is just a way of conversation between a user and a chatbot on how it would lead to a logical end. In general, all chatbots are designed to help users with a set of predefined things; stories just represent how they are done.

We'll try to prepare some simple dialogs in the format that Rasa expects. These dialogs will be stateless—that is, they do not depend on previous dialogs. We will be using our handmade stateless stories for interactive learning.

We'll take a couple of minutes to just hand-curate a few stories that we know of, so that we get acclimatized with how story data is created.

Let's first create a file named `stories.md` in our **data** folder.

```
## story_001
* greeting
  - utter_greet
* get_horoscope
  - utter_ask_horoscope_sign
* get_horoscope{"horoscope_sign": "Capricorn"}
  - slot{"horoscope_sign": "Aries"}
  - get_todays_horoscope
  - utter_subscribe
## story_002
* greeting
  - utter_greet
* get_horoscope{"horoscope_sign": "Capricorn"}
  - slot{"horoscope_sign": "Cancer"}
  - get_todays_horoscope
  - utter_subscribe
* subscription
  - slot{"subscribe": "True"}
  - subscribe_user
## Horoscope query with horoscope_sign

* greeting
  - utter_greet
* get_horoscope
  - utter_ask_horoscope_sign
* get_horoscope{"horoscope_sign": "capricorn"}
  - slot{"horoscope_sign": "capricorn"}
  - get_todays_horoscope
  - slot{"horoscope_sign": "capricorn"}
  - utter_subscribe
* subscription{"subscribe": "True"}
  - slot{"subscribe": "True"}
  - subscribe_user
  - slot{"subscribe": true}
## Horoscope with sign provided
```

```

* greeting
  - utter_greet
* get_horoscope{"horoscope_sign": "leo"}
  - slot{"horoscope_sign": "leo"}
  - get_todays_horoscope
  - slot{"horoscope_sign": "leo"}
  - utter_subscribe
* subscription{"subscribe": "True"}
  - slot{"subscribe": "True"}
  - subscribe_user
  - slot{"subscribe": true}
## When user directly asks for subscription
* greeting
  - utter_greet
* subscription{"subscribe": "True"}
  - slot{"subscribe": "True"}
  - subscribe_user
  - slot{"subscribe": true}

```

If you stare at the stories for a couple of minutes, they will confess to you what they means; it shouldn't be difficult to make some sense of what is happening. The main difference between the first two stories is that in the first story the user doesn't mention his horoscope sign and the bot is supposed to ask the horoscope sign and then proceed with the story.

In the second story, the user himself tells the horoscope sign and then ends the story with the subscribe dialog. We have added couple of more stories covering more use-cases. Feel free to add your own stories in the same file.

So, basically stories are markdown files where we can write as many stories as needed in the markdown format shown earlier. Doing this seems a bit of a tough task by hand. So, we'll try to learn how to use Rasa's own **interactive learning** tool to generate more stories like this.

Let's get started.

Interactive Learning

We have been talking about interactive learning in bits and pieces so far, but it's time to actually write some code and do it. Interactive learning is one of the coolest features Rasa has where it makes the ML part fun and easy. There are two parts: in the first part, we train a model by giving our initial dataset using various policies, and in the second part we test the model, correct it, and re-train it in an interactive manner.

Training the Chatbot Agent Model

Let's create a new file called `train_initialize.py` in our main project directory. Contents of `train_initialize.py` looks like this:

```

from __future__ import absolute_import
from __future__ import division
from __future__ import print_function
from __future__ import unicode_literals
from rasa_core import utils
from rasa_core.agent import Agent

```

```

from rasa_core.policies.keras_policy import KerasPolicy
from rasa_core.policies.memoization import MemoizationPolicy
from rasa_core.policies.sklearn_policy import SklearnPolicy
if __name__ == '__main__':
    utils.configure_colored_logging(loglevel="DEBUG")
    training_data_file = './data/stories.md'
    model_path = './models/dialogue'
    agent = Agent("horoscope_domain.yml",
                  policies=[MemoizationPolicy(), KerasPolicy()])
    training_data = agent.load_data(training_data_file)
    agent.train(
        training_data,
        augmentation_factor=50,
        epochs=500,
        batch_size=10,
        validation_split=0.2
    )
    agent.persist(model_path)

```

This is the code that we write in `train_initialize.py` file. Now, before we move on to the next code file, let's first try to understand important points in it.

1. First, we import a few methods from `__future__` module. Future statements in Python have special usage, they change how your Python module is parsed, and they change the way your existing method behaves.

Curious personality ? Try the below code in your python interpreter

```

from __future__ import braces

```

2. Import `utils` method from `rasa_core` modules to configure logging.
3. Import `agent` class from `agent` module to create the agent object.
4. `KerasPolicy`, `MemorizationPolicy` will be passed as policies parameters to agent class.
5. **configure_colored_logging**: Utility method defined in `utils.py` for colored logging using Python's `coloredlogs` package.
6. **Agent**: A class defined by Rasa that provides an interface to make use of most important Rasa Core functionality, such as training, handling messages, loading a dialog model, getting the next action, and handling a channel.
7. **load_data**: loads training data from the given path.
8. **train**: trains the given policies/policy ensemble using data from the file provided.
9. **training_data**: object returned by **load_data** method. List of `DialogueStateTracker`. This is nothing but our training data file.
10. **augmentation_factor**: tells Rasa how many dummy stories should be created given our initial set of stories. 10x factor is a heuristic for augmentation rounds for training data generator.
11. **epochs**: 1 epoch is a complete training cycle on an entire training dataset. Total number of forward and backward passes of training data.
12. **batch_size**: tells you the amount of training sample to use in each pass. 100 examples with a `batch_size` of 10 will take 10 epochs to go over the entire dataset.
13. **validation_split**: percentage of data to validate the unbiased accuracy of a model.
14. **persist**: this method is used to persist the agent object in a given directory for re-use.

At this point you should be pretty much clear on what each method does and what's happening inside the code.

Before we go on to run the script, make sure you have `rasa_core` library installed before executing this script.

You can install `rasa_core` using the following command:

```
pip install rasa_core==0.11.1
```

If you are following the chatbot example in this book, then make sure you installed the aforementioned version only, as Rasa may not be backward-compatible. They are rapidly coming up with newer and more optimized methods.

The Latest `rasa_core`

You can also install the bleeding edge version of `rasa_core` from the github repo. You just have to execute the following set of commands, which will take the latest code from github directly before installing.

```
git clone https://github.com/RasaHQ/rasa_core.git
cd rasa_core
pip install -r requirements.txt
pip install -e .
```

Let's try to run this code file to train our model as per the given parameters.

```
$python train_initialize.py
```

You can also run this script from Jupyter Notebook itself using Jupyter's magic command, as shown here:

```
!python train_initialize.py #Use python3 if you have installed rasa
for python3
```

It should take about 25 to 30 seconds for the model to be trained on such a small dataset as ours. I added `SklearnPolicy` in the list of policies along with `MemorizationPolicy` and `KerasPolicy` to train my model. Different policies have their own benefits. Read more about them to know which may work better for your use-case; for my dataset `SklearnPolicy` seems to be performing better than the `KerasPolicy` at times.

After the script is done executing you should see some successful messages like these:

```
2018-08-30 04:24:31 INFO rasa_core.policies.keras_policy - Done fitting keras policy model
2018-08-30 04:24:31 INFO rasa_core.featurizers - Creating states and action examples from collec
Processed trackers: 100%|██████████| 96/96 [00:00<00:00, 898.31it/s, # actions=75]
2018-08-30 04:24:31 INFO rasa_core.featurizers - Created 75 action examples.
2018-08-30 04:24:31 INFO rasa_core.policies.sklearn_policy - Done fitting sklearn policy model
2018-08-30 04:24:31 INFO rasa_core.agent - Model directory models/nlu exists and contains old mo
2018-08-30 04:24:31 INFO rasa_core.agent - Persisted model to '/Users/sumit/apress_all/Chapter I
```

You will also find a couple of folders created as per their model name. Make sure you have them in the `model_path` you gave in the script. Below are the folders/files I see inside my `model_path` folder.

```
policy_0_MemoizationPolicy
policy_1_KerasPolicy
policy_2_SklearnPolicy
domain.json
domain.yml
Policy_metadata.json
```

If you have validated that your model has successfully completed the execution and has persisted the model in your local system, then we are good to go to the next step of interactive training.

Real-Time Training by Reinforcement

In this section we are going to write some more code to train our dialog model and retrain when it gives incorrect output.

So, when our bot does something wrong we immediately jump in and let the model know that its prediction is wrong by telling it what is right. Without having to stop, the model retrains itself, and once we are done, the interaction between the user and the bot gets captured to a file and added to our existing training data. It works more like a feedback system in every step, rather than waiting for a single reward at the end.

The next step is to create a new file called `endpoints.yml` with the below contents. We'll be using this file in our Python code file `train_online.py`. With this configuration we can expose the Rasa method as HTTP APIs.

```
action_endpoint:
  url: http://localhost:5055/webhook
#nlg:
# url: http://localhost:5056/nlg
core_endpoint:
  url: http://localhost:5005
```

Now, let's create `train_online.py` for our online/interactive training purposes.

```
from __future__ import absolute_import
from __future__ import division
from __future__ import print_function
from __future__ import unicode_literals

import logging
from rasa_core import utils, train
from rasa_core.training import online
from rasa_core.interpreter import NaturalLanguageInterpreter
logger = logging.getLogger(__name__)
def train_agent(interpreter):
    return train.train_dialog_model(domain_file="horoscope_domain.yml",
```

```

stories_file="data/stories.md",
output_path="models/dialog",
nlu_model_path=interpreter,
endpoints="endpoints.yml",
max_history=2,
kwargs={"batch_size": 50,
        "epochs": 200,
        "max_training_samples": 300
      })

if __name__ == '__main__':
    utils.configure_colored_logging(loglevel="DEBUG")
    nlu_model_path = "./models/nlu/default/horoscopebot"
    interpreter = NaturalLanguageInterpreter.create(nlu_model_path)
    agent = train_agent(interpreter)
    online.serve_agent(agent)

```

`max_history` is the number of states to keep track of by the model.

Before we go on to run our final script, `train_online.py`, we should know and make ourselves ready for something called `rasa-nlu-sdk`.

rasa-nlu-sdk

Rasa NLU stack comes up with `rasa-nlu-sdk`, which is a Python SDK for the development of custom actions for Rasa Core. As for our chatbot example, we need to define a few custom actions, like hitting the API to get today's horoscope or maybe a database writing operations to add the user to the subscribe list.

The good news is they have a separate library for this, and we can install that using `pip`.

Let's install that using the following command:

```
pip install rasa-core-sdk==0.11.0
```

Now, we need to go to another tab of our terminal or a fresh command line and execute the following command in our project directory (the place where our `actions.py` file is):

```

python -m rasa_core_sdk.endpoint --actions actions
INFO:__main__:Starting action endpoint server...
INFO:rasa_core_sdk.executor:Registered function for 'get_todays_horoscope'.
INFO:rasa_core_sdk.executor:Registered function for 'subscribe_user'.
INFO:__main__:Action endpoint is up and running. on ('0.0.0.0', 5055)

```

This command will start an action server that will be listening to any custom action predicted by the model. As soon as any action is triggered, it will execute it and give the response as per the method.

The default port for the action server endpoint is 5055 on localhost. If you want to change it, you can add pass `--port` argument in the command line.

A quick question comes to mind: why? Why do I need a separate server for that? Why can't we use plain Python? Yes, we can use plain Python,

but suppose you were to develop the required actions in any other language or you have some actions already exposed as APIs. Now, you just need to go to `endpoints.yml`, which we already created, and use that to mention from where your action server should be used and where your `core_endpoint` server should be. In a production system both can be different servers having altogether different urls.

Now, when we run our next script where we mentioned our `endpoints.yml` file, Rasa will read the file and get the configuration for our `action_server`, which is already up and running as per the configuration we did earlier.

```
action_endpoint:
  url: http://localhost:5055/webhook
```

Let's run the `train_online.py` in a fresh command line terminal.

```
$python3 train_online.py
```

After a successful training of the dialog model, we'll be getting a message like this:

```
2018-08-30 07:09:37 INFO      rasa_core.policies.keras_policy - Done fitting keras policy model
2018-08-30 07:09:37 INFO      rasa_core.agent - Model directory models/nlu exists and contains old m
2018-08-30 07:09:37 INFO      rasa_core.agent - Persisted model to '/Users/sumit/apress_all/Chapter
2018-08-30 07:09:37 INFO      rasa_core.training.online - Rasa Core server is up and running on http
Bot loaded. Type a message and press enter (use '/stop' to exit).
127.0.0.1 - - [2018-08-30 07:09:37] "GET /domain HTTP/1.1" 200 996 0.001847
```

Now, here you can start talking to your bot that has just been born. From here it's solely in your hands how you train it. You can correct it whenever it responds something bad or unexpected.

Let's do that and see if it improves.

I enter "Hi" as the first message and the bot returns the following:

```
-----

Chat history:

bot did: action_listen
user said: hi
whose intent is: {'confidence': 0.8472929307505297, 'name': 'greeting'}

we currently have slots: DD: None, MM: None, horoscope_sign: None, subscribe: Non

-----

The bot wants to [utter_greet] due to the intent. Is this correct?

1.      Yes
2.      No, intent is right but the action is wrong
3.      The intent is wrong
0.      Export current conversations as stories and quit
```

Now, the bot is telling me what it did based on what the user said and how he understands it. Now, it also gives us four options. We'll talk about that in a moment.

What **bot_did** here is `action_listen` meaning; it was just waiting for the user to enter something, which is fine.

Bot says the intent `{'confidence': 0.8472929307505297, 'name': 'greeting'}`, which is correct, so we'll press 1, meaning what the bot is understanding and trying to do it correct.

```

1
-----
Chat history:
    bot did: action_listen
    user said: hi
    whose intent is: {'confidence': 0.8472929307505297, 'name': 'greeting'}
    bot did: utter_greet
we currently have slots: DD: None, MM: None, horoscope_sign: None, subscribe: None
-----
The bot wants to [action_listen]. Is this correct?
1.      Yes.
2.      No, the action is wrong.
0.      Export current conversations as stories and quit
-----

```

And there you are, our bot at least greeted the user properly. If you see the aforementioned response, you can actually check what our bot did; it keeps track of all this while we train it.

```
bot did: utter_greet
```

Now, the bot says that the next thing is to wait and listen to what the user is going to say (i.e., waiting for the user input). Is this correct? Yes, that's correct, so we press 1 again and the bot gives a prompt to the user.

I enter, "What's my horoscope for today?"

```

-----
Chat history:
    bot did: action_listen
    user said: hi
        whose intent is: {'confidence': 0.8472929307505297, 'name': 'greeting'}
    bot did: utter_greet
    bot did: action_listen
    user said: what's my horoscope for today?
        whose intent is: {'confidence': 0.8902154738608781, 'name': 'get_horoscope'}
we currently have slots: DD: None, MM: None, horoscope_sign: None, subscribe: None
-----
The bot wants to [utter_ask_horoscope_sign] due to the intent. Is this correct?
1.      Yes
2.      No, intent is right but the action is wrong
3.      The intent is wrong
0.      Export current conversations as stories and quit
-----

```

Now, the bot very correctly identifies the intent to be “get_horoscope” intent with 89 percent accuracy, which is quite good. The bot also says that it wants to **utter_ask_horoscope_sign**, which is correct again because the user has not mentioned any horoscope sign yet and the slot is still None, as shown.

We press 1 again.

```
-----
Chat history:
  bot did: action_listen
  user said: hi
  whose intent is: {'confidence': 0.8472929307505297, 'name': 'greeting'}
  bot did: utter_greet
  bot did: action_listen
  user said: what's my horoscope for today?
  whose intent is: {'confidence': 0.8902154738608781, 'name': 'get_horoscope'}
  bot did: utter_ask_horoscope_sign
we currently have slots: DD: None, MM: None, horoscope_sign: None, subscribe: None
-----
The bot wants to [action_listen]. Is this correct?
  1.      Yes.
  2.      No, the action is wrong.
  0.      Export current conversations as stories and quit
-----
```

Now, the bot asks if it's time to wait for the user input. Yes, the user needs to provide the horoscope sign.

We press 1 again, and when prompted, we entered “Capricorn” as the input.

```
-----
Chat history:
  bot did: action_listen
  user said: hi
  whose intent is: {'confidence': 0.8472929307505297, 'name': 'greeting'}
  bot did: utter_greet
  bot did: action_listen
  user said: what's my horoscope for today?
  whose intent is: {'confidence': 0.8902154738608781, 'name': 'get_horoscope'}
  bot did: utter_ask_horoscope_sign
  bot did: action_listen
  user said: Capricorn
  whose intent is: {'confidence': 0.8456705103362621, 'name': 'get_horoscope'}
  with horoscope_sign: capricorn
we currently have slots: DD: None, MM: None, horoscope_sign: capricorn, subscribe: None
The bot wants to [get_todays_horoscope] due to the intent. Is this correct?
  1.      Yes
  2.      No, intent is right but the action is wrong
  3.      The intent is wrong
  0.      Export current conversations as stories and quit
-----
```

Now, as we can see, the horoscope_sign slot is automagically set above to “**Capricorn.**”

The next step is to teach the bot to call `get_todays_horoscope` and to call the `get_todays_horoscope` action. By this time our slot is already set for the `horoscope_sign` variable that we defined in `horoscope_domain.yml`, so if this action is called again, the chatbot should return the actual horoscope for the day. Let's try.

I entered "Capricorn."

The bot hits the action server's endpoint and brings the result back to the user as we defined in our custom action.

Your today's horoscope:

The journey from defeat to success is driven by optimism. You will be very optimistic today—about your work, future, and everything that can lead you to success, says Ganesha. You will also be very careful while dealing with day-to-day work, and will capitalize on every opportunity that can get you a step closer to your goals. Determination and good luck is all you need to realize your dreams; you will have both today.

This seems fair, as I am missing my deadline to complete this chapter, and right now it's 6:00 AM in the morning and I have not slept yet. Doing every bit so that your money is worth spending on this book.

Okay, let's get back to see what our bot is trying to do next.

```
-----
The bot wants to [action_listen]. Is this correct?
1.      Yes.
2.      No, the action is wrong.
0.      Export current conversations as stories and quit
```

Maybe we don't want our bot to listen now after it has uttered the horoscope for today. We want our bot to ask for the user to subscribe to the updates as per our conversation script and also as per the stories we created.

So, I would press 2, and it says

```
-----
what is the next action for the bot?
0          action_listen    1.00
1          action_restart   0.00
2          action_default_fallback 0.00
3          utter_greet      0.00
4          utter_ask_horoscope_sign 0.00
5          utter_ask_dob     0.00
6          utter_subscribe   0.00
7          get_todays_horoscope 0.00
8          subscribe_user    0.00
```

Since we want our user to subscribe, we should press 6 so that the bot asks the user if he/she wants to subscribe for daily updates.

The bot replies, “Do you want to subscribe for daily updates?”

Now the bot says:

```
-----
The bot wants to [get_todays_horoscope]. Is this correct?
1.      Yes.
2.      No, the action is wrong.
0.      Export current conversations as stories and quit
```

The action here is not correct, so we press 2 and train the bot to listen to the user’s action here.

```
-----
what is the next action for the bot?
0          action_listen    0.21
1          action_restart   0.01
2          action_default_fallback 0.01
3          utter_greet      0.21
4          utter_ask_horoscope_sign 0.51
5          utter_ask_dob     0.01
6          utter_subscribe   0.02
7          get_todays_horoscope 0.78
8          subscribe_user    0.22
-----
```

We press 0 when the bot is waiting to know the correct next step. The bot will now wait for the user input in our interactive learning process.

The user says here, “Please subscribe me,” and our bot successfully with 80 percent confidence recognizes this to be a subscription intent and also, based on our training data in data.json, understands pretty easily that the user is asking to subscribe and not unsubscribe. In other words, it sets the slot of subscribe to True, as you can see here:

```
-----
user said: Please subscribe me
whose intent is: {'confidence': 0.795172441763619, 'name': 'subscription'}
with subscribe: True
we currently have slots: DD: None, MM: None, horoscope_sign: capricorn, subscribe: True
-----
```

Now, that our bot has understood the intent and also parsed the entities in it, it’s time to reply back to the user saying, “He is successfully subscribed,” and bingo—we get a message from bot:

```
You're successfully subscribed
```

There you go! You can successfully build your own in-house chatbot all by yourselves. As a part of the exercise, I would recommend you add all use-cases we discussed for HoroscopeBot and see if it works. Try to add more uses-cases and more functionality when everything we have done so far works well.

You can add more utterance messages here for the bot to say, “Goodbye, have a good day,” after the conversation is completed. I strongly believe you should be in a position to do that. Make sure you do that and let us know how it went for you.

Now, the final part is to give all of this learning back to the user so that we don’t have to teach the same stories again and the bot tries to learn from the existing stories and more to come.

Exporting Conversations As Stories

Now the next step in the interactive learning part is to press 0 and export our current conversations as stories and quit.

We press 0 for that and the bot prompts something like this:

```
File to export to (if file exists, this will append the stories) [stories.md]:
```

Just enter our **stories.md** filename, if you have kept any other name, then make sure you give the correct filename.

And Rasa does the hard job for us and generates a stories.md file for us automatically again like this:

```
## Generated Story 3797421409943253925
* greeting
  - utter_greet
* I want to know my horoscope
  - utter_ask_horoscope_sign
* get_horoscope
## Generated Story 7304374603374920907
* greeting
  - utter_greet
* what is my todays horoscope?
  - utter_ask_horoscope_sign
* Cancer
## Generated Story -6877464862083507836
* greeting
  - utter_greet
* get_horoscope
  - utter_ask_horoscope_sign
* get_horoscope{"horoscope_sign": "capricorn"}
  - slot{"horoscope_sign": "capricorn"}
  - get_todays_horoscope
  - slot{"horoscope_sign": "capricorn"}
  - utter_subscribe
* subscription{"subscribe": "True"}
  - slot{"subscribe": "True"}
  - subscribe_user
  - slot{"subscribe": true}
```

You can either change the source of the stories file while training the model with the aforementioned file or you can copy and paste the newly generated stories into our original stories.md file. For now, I am going to copy and paste these stories in our original stories.md file. Make sure to run your models again before testing or doing interactive learning again.

Testing the Bot

Testing the bot is a part of our interactive learning process. We keep checking all the possible scenarios initially thought for the bot and train it accordingly. I tested Horoscope Bot for some other scenarios and trained it accordingly. The following is one of them that is a bit different than what we did earlier in this chapter.

Test Case 1

Let's run the first test:

- **User:** Hello there!
- **HoroscopeBot:** Hello! What can I do for you?
- **User:** What's the horoscope for Capricorn today?
- **HoroscopeBot:** Your today's horoscope:

The journey from defeat to success is driven by optimism. You will be very optimistic today—about your work, future, and everything that can lead you to success, says Ganesha. You will also be very careful while dealing with day-to-day work, and will capitalize on every opportunity that can get you a step closer to your goals. Determination and good luck is all you need to realize your dreams; you will have both today.

- **HoroscopeBot:** Do you want to subscribe for daily updates?
- **User:** Please subscribe me.
- **HoroscopeBot:** You're successfully subscribed.

As you can see in this conversation between the user and the chatbot, we tested if the bot handles the scenario where the user himself provides the horoscope_sign while asking for the horoscope for the day itself. The bot does well in this case and ends up asking for a subscription and, based on the user response, takes appropriate action.

You should do a sanity testing of the bot's conversation to determine if normal flow is working or not.

I also tested the story we mentioned in our stories.md file when a user directly asked for a subscription and it worked as well.

Test Case 2

Now the second test:

- **User:** Hi
- **HoroscopeBot:** Hello! What can I do for you?
- **User:** Subscribe me please for daily updates.
- **HoroscopeBot:** You're successfully subscribed.

I will be adding a couple of more new stories as well so that the bot works flawlessly. But as of now we have a fully working bot. In the second version of the code on github you will also find the bot to be correcting the spelling mistakes for horoscope sign, retrieving horoscope sign based on user's DOB, saying a goodbye message, etc. I strongly recommend you check the code on how it is done and provide your feedback. But before

that, give a thought to how it'll be done and where the changes will go. We have knowingly not mentioned the code for the other use-cases in this chapter, so that it doesn't become fussy while you are concentrating on learning the tricks of the trade.

You can download the latest Python code and Jupyter Notebooks from our github repo and try to install the correct packages to run it. You will find more use-cases handled in the code as discussed in this chapter.

Summary

In this chapter, we learned about Rasa-NLU and why Rasa-NLU is better than any other open-source tool available on the market. We learned how to configure pipelines by using tensorflow, sklearn, and keras in our pipeline configurations.

We learned to create everything from scratch on our local system without being dependent on any services requiring you to use their APIs, like Dialogflow, wit.ai, etc.

We also learned how to create stories and how to train an NLU model and a dialog model and use them both using Rasa Core to build a bot by training using the coolest feature, interactive learning. We also got a fair idea of how to create training data easily and annotate that easily with help of the open-source tools like rasa-nlu-trainer. I hope this chapter was more interactive for you than any other chapter. If you are not having a feeling of achievement, then brace yourself for the next chapter, where we'll be actually taking it live to our audiences and showing the world what bots are capable of. We'll be learning to integrate the chatbot from this chapter to various platforms like Facebook and Slack using our own web servers.

Keep training your bot until in the next chapter we take it live.

See you in the next chapter.